

Reconeixement estructural

Objectiu de la classe: Analitzar i aprofundir en la resolució de problemes de reconeixement de patrons basat en l'estructura.

Reconeixement de patrons

Problema 1: Es vol fer un sistema per ensenyar a escriure a nens. Per aquest objectiu, s'està treballant en una interfície d'entrada en la que el nen pot escriure una lletra, i el sistema ha d'identificar quina lletra de l'alfabet ha escrit.

Per fer aquest sistema es vol fer servir una **representació estructural de les lletres** que permeti fer un seguiment exacte del que escriu el nen i poder-li explicar on s'equivoca, i així anar supervisant el seu progrés.

Inicialment es treballarà amb 26 lletres majúscules de l'alfabet llatí:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

es proposa treballar amb grafs que tenen les següents propietats:

- Etiquetes en els nodes
- Arcs no dirigits
- Sense etiquetes en els arcs

Reconeixement de patrons

Etiquetes del Nodes: es proposa fer servir 8 tipus diferents d'etiquetes pels nodes, en funció del tipus de vèrtex que representa.

Hi ha moltes possibilitats diferents, es proposen les següents:

Tipus 1: Vèrtex terminal (llarg)



Tipus 2: Vèrtex terminal (curt)



Tipus 3: Contorn arrodonit



Tipus 4: Vèrtex intersecció de 2 arestes (angle gran)



Tipus 5: Vèrtex intersecció de 2 arestes (angle petit)



Tipus 6: Vèrtex intersecció de 3 arestes (tipus T)



Tipus 7: Vèrtex intersecció de 3 arestes (tipus Y)

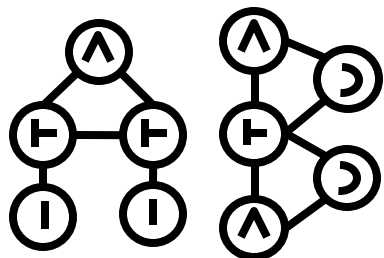


Tipus 8: Vèrtex intersecció de 4 arestes

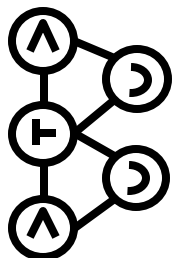


Reconeixement de patrons

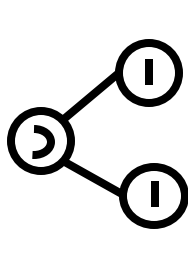
Possible Alfabet: es proposa fer servir el següent alfabet



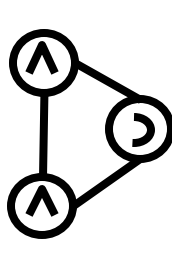
A



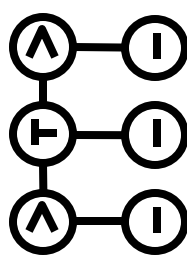
B



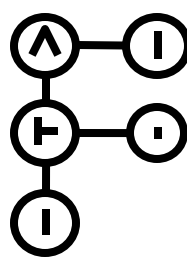
C



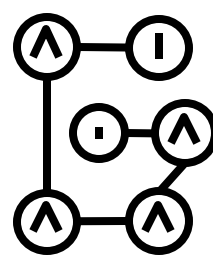
D



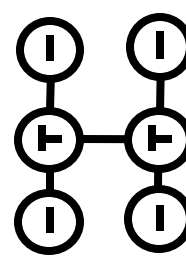
E



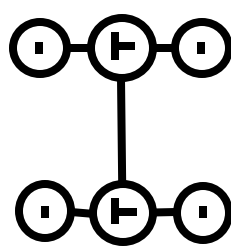
F



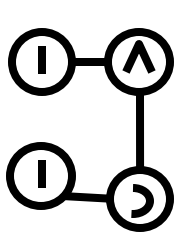
G



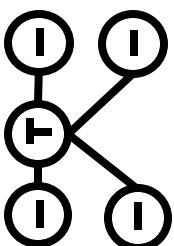
H



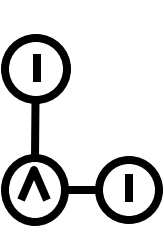
I



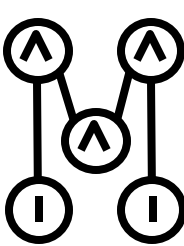
J



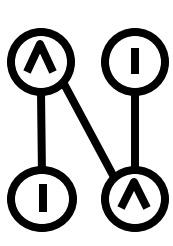
K



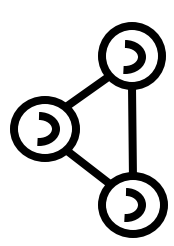
L



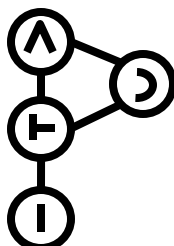
M



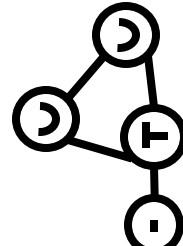
N



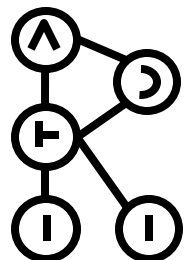
O



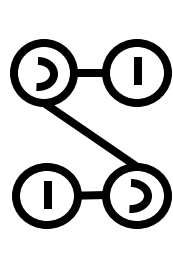
P



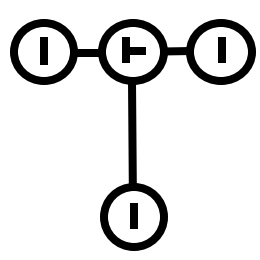
Q



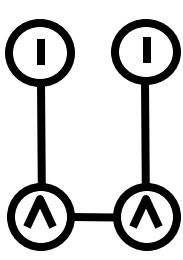
R



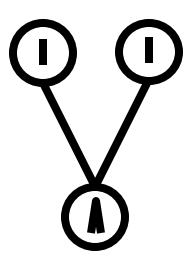
S



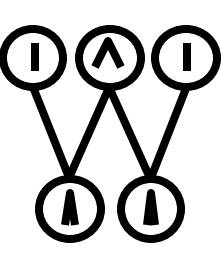
T



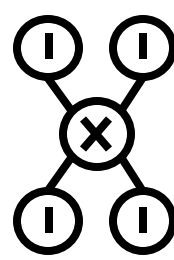
U



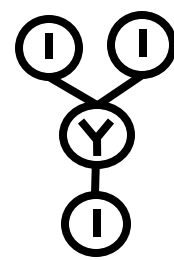
V



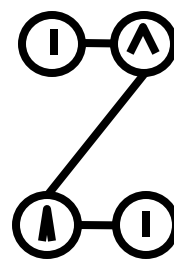
W



X



Y



Z

Reconeixement de patrons

Es demana que construïu un algorisme Reconeixedor que faci el següent:

Paràmetres:

- Graf d'entrada: graf de la lletra que ha escrit el nen a la pantalla
(*graf que rebem del sistema de visió que captura el dibuix de la pantalla*)
- Alfabet: depèn del nivell de dificultat que té l'usuari, pot canviar.

Retorna: lletra identificada o llista de lletres més probables

Funció Reconeixedor (<graf>,<alfabet>)

1. Cercar dins de l'alfabet, la lletra que correspon a <graf>.
2. **Si** (hi ha una correspondència exacta) llavors
 - Retornar(la lletra de la correspondència exacta)
3. **Sinó**
 - Cercar les N primeres lletres de l'alfabet amb distància d'edició més petita.

4. **fSi**

fFunció

Reconeixement de patrons

Sobre el Punt 1 de l'algorisme:

Funció Reconeixedor (<graf>)

1. Cercar dins de l'alfabet, la lletra que correspon a <graf>.
2. Si (hi ha una correspondència exacta) llavors
 - Retornar(la lletra de la correspondència exacta)
3. Sinó
 - Cercar les N primeres lletres de l'alfabet amb distància d'edició més petita.
4. fSi

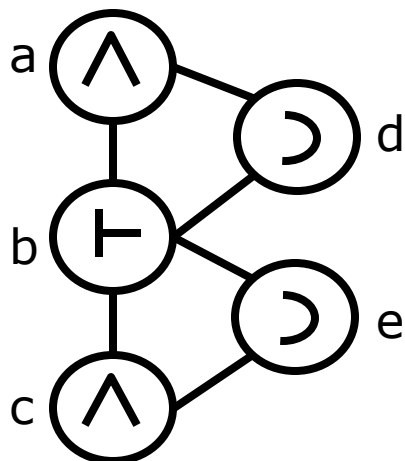
fFunció

Contesteu a les següents preguntes:

1. Escriu la matriu d'adjacència que pot representar una lletra, fes-ho pel cas de la lletra B. És simètrica?, perquè?
2. Quin algorisme faries servir per implementar el pas (1) de l'algorisme?
3. Quina serà la complexitat d'aquest algorisme en el pitjor cas?
4. Es pot aplicar alguna millora tipus AC?

Reconeixement de patrons

1. Escriu la matriu d'adjacència que pot representar una lletra, fes-ho pel cas de la lletra B. És simètrica?, perquè?



Etiqueta
nodes

$$MA(B) = \begin{matrix} & \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

	Etiqueta
	1
	2
	3
	4
	5
	6
	7
	8

és simètrica? **SÍ**

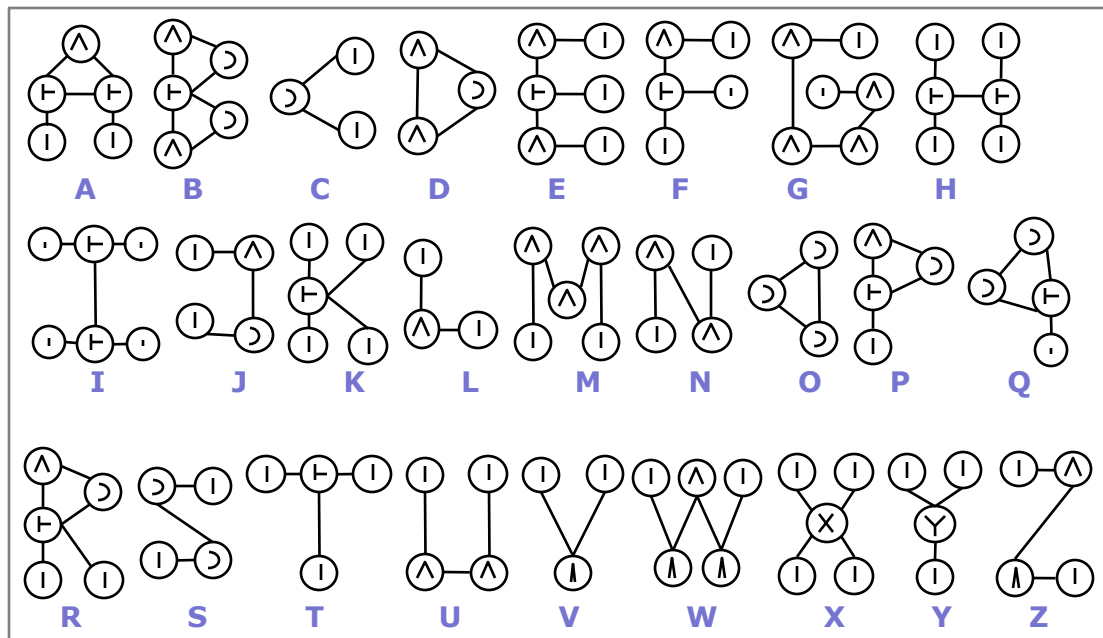
Perquè? **perquè el graf és no dirigit**

2. Quin algorisme faries servir per implementar el pas (1) de l'algorisme?

Graph-Matching(<graf>, <x ∈ Alfabet>)

Reconeixement de patrons

3. Quina serà al complexitat d'aquest algorisme en el pitjor cas, per un alfabet com el de la figura? (a la taula teniu el número de nodes que té cada lletra)



5 lletres de 3 nodes

9 lletres de 4 nodes

8 lletres de 5 nodes

4 lletres de 6 nodes

Lletra	#nodes
C	3
D	3
L	3
O	3
V	3
J	4
N	4
P	4
Q	4
S	4
T	4
U	4
Y	4
Z	4
A	5
B	5
F	5
K	5
M	5
R	5
W	5
X	5
E	6
G	6
H	6
I	6

Càlcul de la complexitat per a cada tipus de lletra:

Lletres de 3 nodes: $5 \cdot (3!) = 5 \cdot 6 = 30$

Lletres de 4 nodes: $9 \cdot (4!) = 9 \cdot 24 = 216$

Lletres de 5 nodes: $8 \cdot (5!) = 8 \cdot 120 = 960$

Lletres de 6 nodes: $4 \cdot (6!) = 4 \cdot 720 = 2880$ **→ Pitjor cas!!!**

Reconeixement de patrons

4. Es pot aplicar alguna millora tipus AC considerant el número d'enllaços per node? (A la taula es donen el nombre d'enllaços per node)

Lletra	#nodes	#Vèrtex / node			
		1	2	3	4
C	3	2	1		
L	3	2	1		
V	3	2	1		
D	3		3		
O	3		3		
P	4	1	2	1	
Q	4	1	3		
J	4	2	2		
N	4	2	2		
S	4	2	2		
U	4	2	2		
Z	4	2	2		
T	4	3		1	
Y	4	3		1	
A	5	2	1	2	
F	5	2	2	1	
R	5	2	2		1
M	5	2	3		
W	5	2	3		
K	5	3	1		1
X	5	4			1
B	5		4		1
G	6	2	4		
E	6	3	2	1	
H	6	4		2	
I	6	4		2	

Sí, considerant la propietat de #vèrtex/node, podem reduir la complexitat. Tot seguit, la calculem en funció del nombre de nodes de cada lletra:

Si la lletra té 3 nodes, hi ha 2 possibilitats:

- $3 \cdot (2! \cdot 1!) = 3 \cdot 2 = 6$
- $2 \cdot (3!) = \mathbf{12}$

abans 30

Si la lletra té 4 nodes:

- $5 \cdot (2! \cdot 2!) = 5 \cdot 2 \cdot 2 = \mathbf{20}$
- $1 \cdot (1! \cdot 2! \cdot 1!) = 1 \cdot 2 = 2$
- $1 \cdot (1! \cdot 3!) = 1 \cdot 6 = 6$
- $2 \cdot (3! \cdot 1!) = 2 \cdot 6 = 12$

abans 216

Si la lletra té 6 nodes:

- $2 \cdot (4! \cdot 2!) = 2 \cdot 48 = \mathbf{96}$
- $1 \cdot (2! \cdot 4!) = 1 \cdot 48 = 48$
- $1 \cdot (3! \cdot 2! \cdot 1!) = 1 \cdot 12 = 12$

abans 2880

Si la lletra té 5 nodes:

- $1 \cdot (2! \cdot 1! \cdot 2!) = 1 \cdot 4 = 4$
- $1 \cdot (2! \cdot 2! \cdot 1!) = 1 \cdot 4 = 4$
- $1 \cdot (2! \cdot 2! \cdot 1!) = 1 \cdot 4 = 4$
- $2 \cdot (2! \cdot 3!) = 2 \cdot 12 = \mathbf{24}$
- $1 \cdot (3! \cdot 1! \cdot 1!) = 1 \cdot 6 = 6$
- $1 \cdot (4! \cdot 1!) = 1 \cdot 24 = \mathbf{24}$

abans 960

Si a més a més féssim servir el tipus de node, encara es podria rebaixar més la complexitat.

Reconeixement de patrons

Sobre el Punt 3 de l'algorisme:

Funció Reconexedor (<graf>)

1. Cercar dins de l'alfabet, la lletra que correspon a <graf>.
2. **Si** (hi ha una correspondència exacta) llavors
 - Retornar(la lletra de la correspondència exacta)
3. **Sinó**
 - Cercar les N primeres lletres de l'alfabet amb distància d'edició més petita.

4. **fSi**

fFunció

Contesteu a les següents preguntes:

1. Quin algorisme aplicaries per calcular la distància d'edició?
2. Què representaria cada branca de l'arbre?
3. Com seria la funció g?, i com la funció h?

Reconeixement de patrons

Contesteu a les següents preguntes:

1. Quin algorisme aplicaries per calcular la distància d'edició?

Com que la distància d'edició és la distància mínima entre dos grafs, aleshores faria servir o bé CCU o bé A*

2. Què representaria cada branca de l'arbre?

L'aplicació d'una operació d'edició:

- Substituir (etiqueta de node)
- Inserir (node o enllaç) o
- Eliminar (node o enllaç).

a cada nivell una operació diferent

Reconeixement de patrons

6. Com seria la funció g ?, i com la funció h ?

La funció g serà la suma dels costos d'edició acumulats pel camí.

$$g(GA) = \sum_m \text{Operacions Edició aplicades}(GI \rightarrow GA)$$

m : Número d'operacions d'edició aplicades

GI: Graf Inicial

GA: Graf Actual representat al node de l'arbre

La funció h es pot calcular a partir del nombre de nodes i el nombre d'enllaços del graf del camí actual que encara no coincideixen amb els nodes del graf objectiu.

$$h(GA) = \sum_{k+v} \min(\text{Costos Operacions Edició})$$

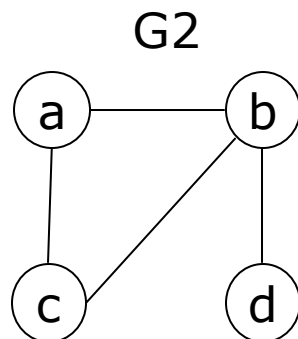
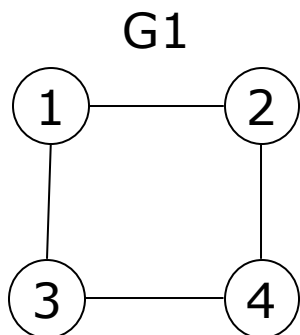
GO: Graf Objectiu

$GO - GA$: Conjunt de nodes del GO que no estan al GA
 $k = \#(GO - GA)$

$VO - VA$: Conjunt de vertex del GO que no estan al GA
 $v = \#(VO - VA)$

Reconeixement estructural

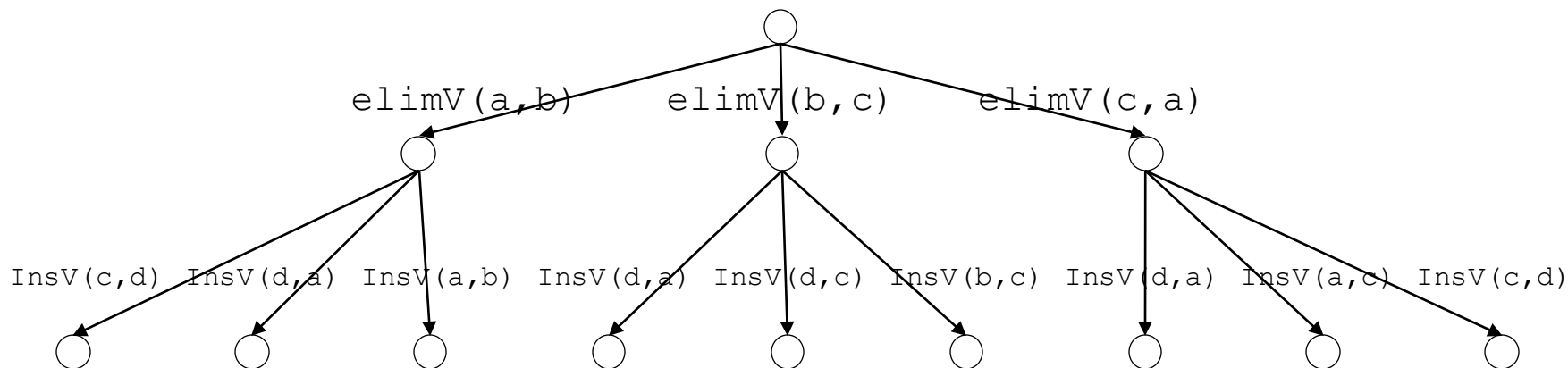
Problema 2: Completa l'arbre de cerca òptima que es dona a sota, en el que es pot trobar la distància d'edició entre els següents dos grafs: G1 i G2.



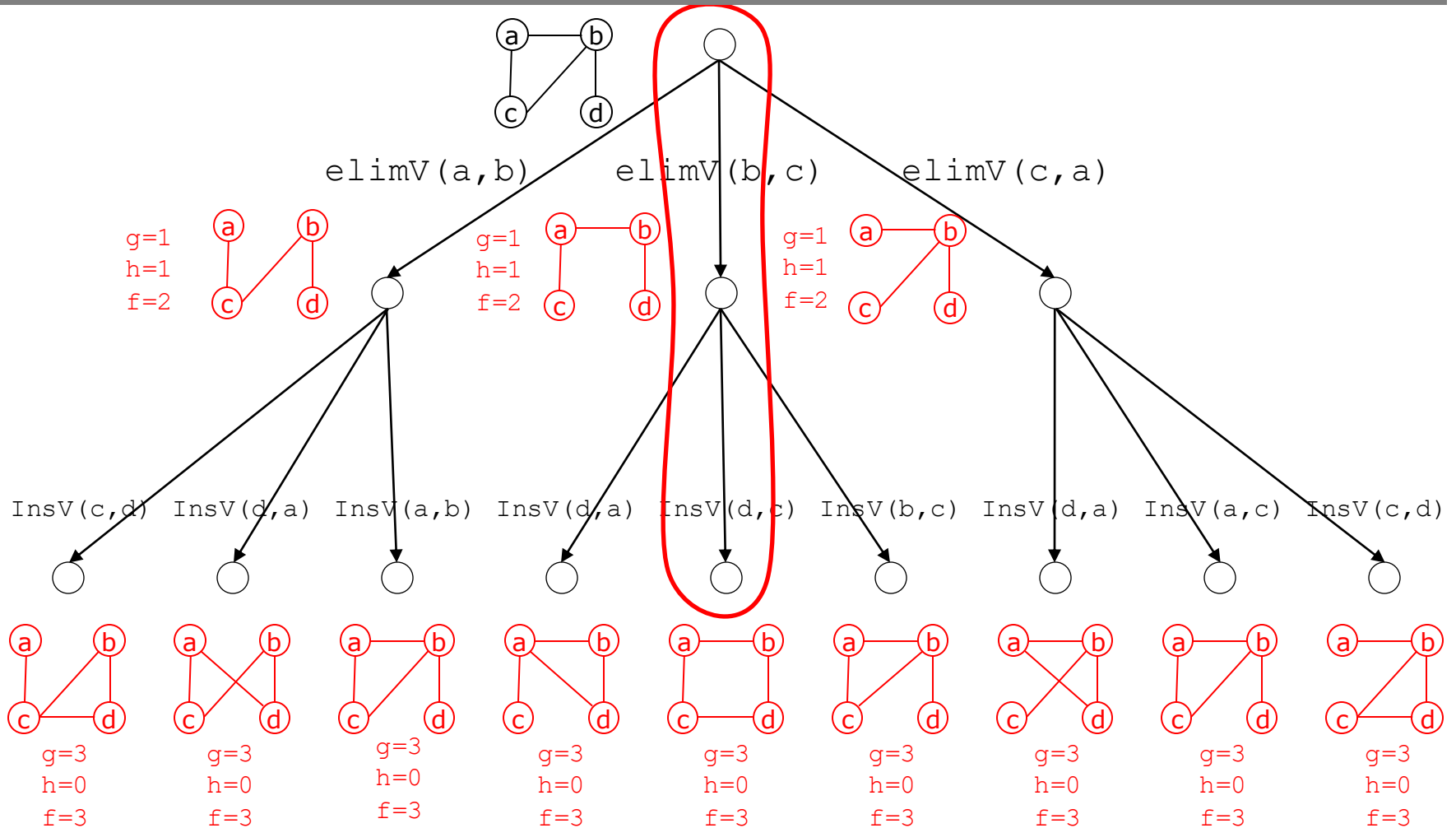
Considerarem que en el primer nivell es fan totes les possibles operacions d'esborrar i en el segon nivell totes les possibles operacions d'inserir nodes, i que les operacions d'edició possible i els seus costos són:

- Cost Eliminar(vèrtex(X,Y)) és 1
- Cost Inserir(vèrtex(X,Y)) és 2

Suposant que apliquem l'A* indica els valors de les funcions: g, h i f. Considera l'heurística definida a l'exercici anterior, i dibuixa el graf associat a cada node de l'arbre. Indica el camí solució i el cost òptim.



Reconeixement estructural



Estat
Objectiu

Resultat: $\text{Esborrar}(\text{vèrtex}(c, b)) + \text{Inserir}(\text{vèrtex}(c, d)) = 3$

Reconeixement de patrons

Problema 3: Calculeu la distància d'edició entre les següents paraules:

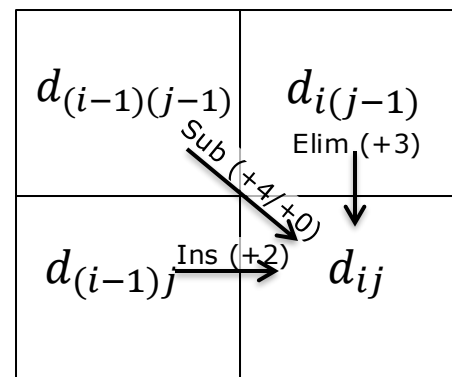
Paraula d'entrada: [STAR]**TREK**

Paraula d'arribada: [STAR]**WARS**

fent servir l'algorisme d'string matching vist a classe.

Recordeu que:

- Cost(eliminació)=3
- Cost(inserció)=2
- Cost(substitució)= $\begin{cases} 0 & \text{si pel mateix caràcter} \\ 4 & \text{si per caràcter diferent} \end{cases}$



$$d_{ij} = \min\{ \underbrace{d_{(i-1)j} + 2}_{\text{inserció}}, \underbrace{d_{(i-1)(j-1)} + 0/4}_{\text{substitució}}, \underbrace{d_{i(j-1)} + 3}_{\text{eliminació}} \}$$

Reconeixement estructural

Pas 1. Construiu la taula de totes les possibles paraules que es poden construir fent servir les operacions d'edició

Pas 2: Calcular els costos de la fila 1

Pas 3: Calcular els costos de la columna 1

Pas 4: Calcular la resta de costos d'esquerra a dreta i de dalt a baix aplicant l'operació de mínim.

Pas 5: Extreure la seqüència d'operacions d'edició que ens porten a la distància d'edició mínima

Reconeixement estructural

Pas 1. Construïu la taula de totes les possibles paraules que es poden construir fent servir les operacions d'edició

		Inserció				
		∅	W	A	R	S
Eliminació	∅	TREK	WTREK	WATREK	WARTREK	WARSTREK
	T	_REK	WREK	WAREK	WARREK	WARSREK
	R	__EK	WEK	WAEK	WAREK	WARSEK
	E	___K	WK	WAK	WARK	WARSK
	K	____	W	WA	WAR	WARS

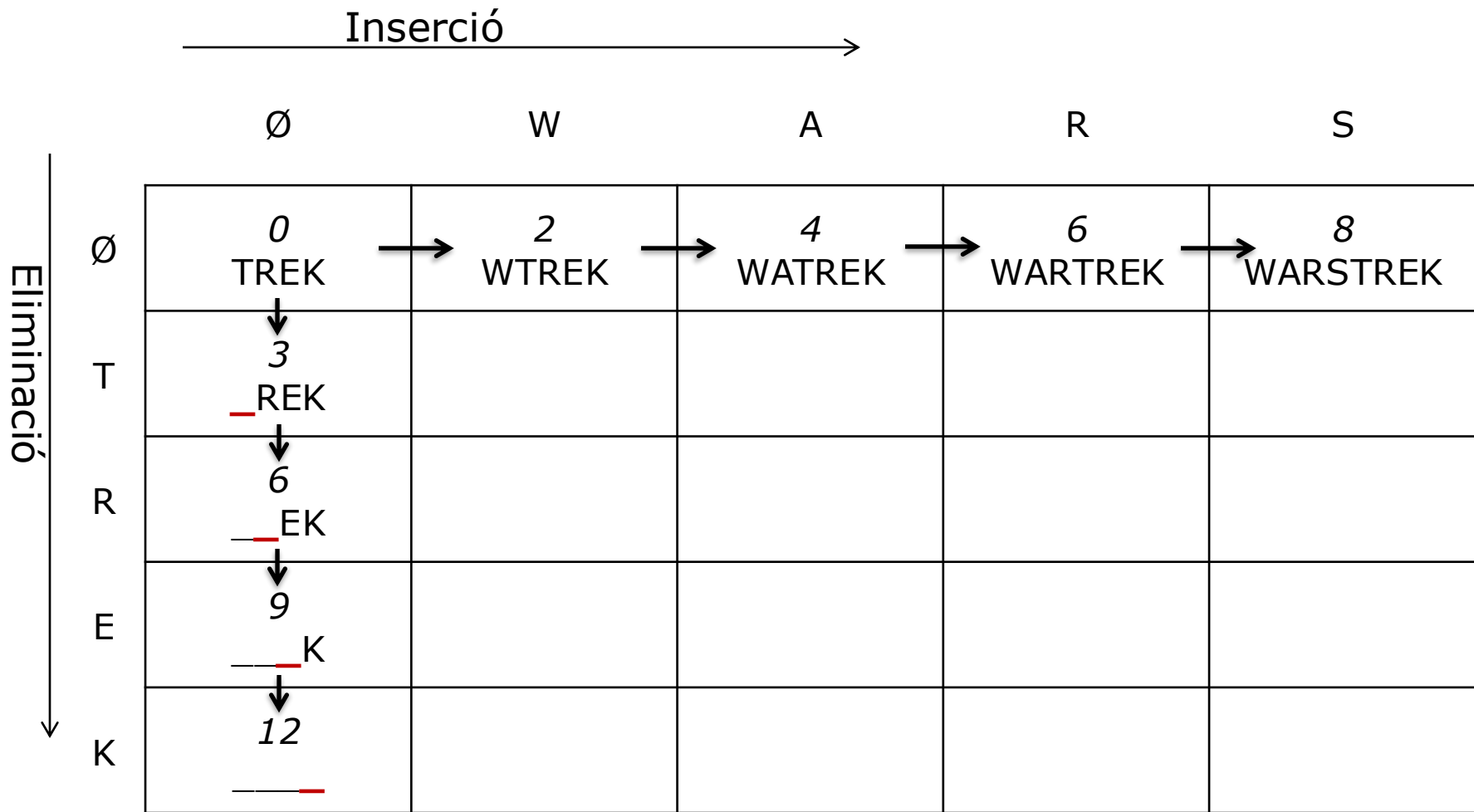
Reconeixement estructural

Pas 2: Calcular els costos de la fila 1

		Inserció →				
		∅	W	A	R	S
Eliminació ↓	∅	0 TREK	→ 2 W T REK	→ 4 W A TREK	→ 6 W A R T REK	→ 8 W A R S TREK
	T					
	R					
	E					
	K					

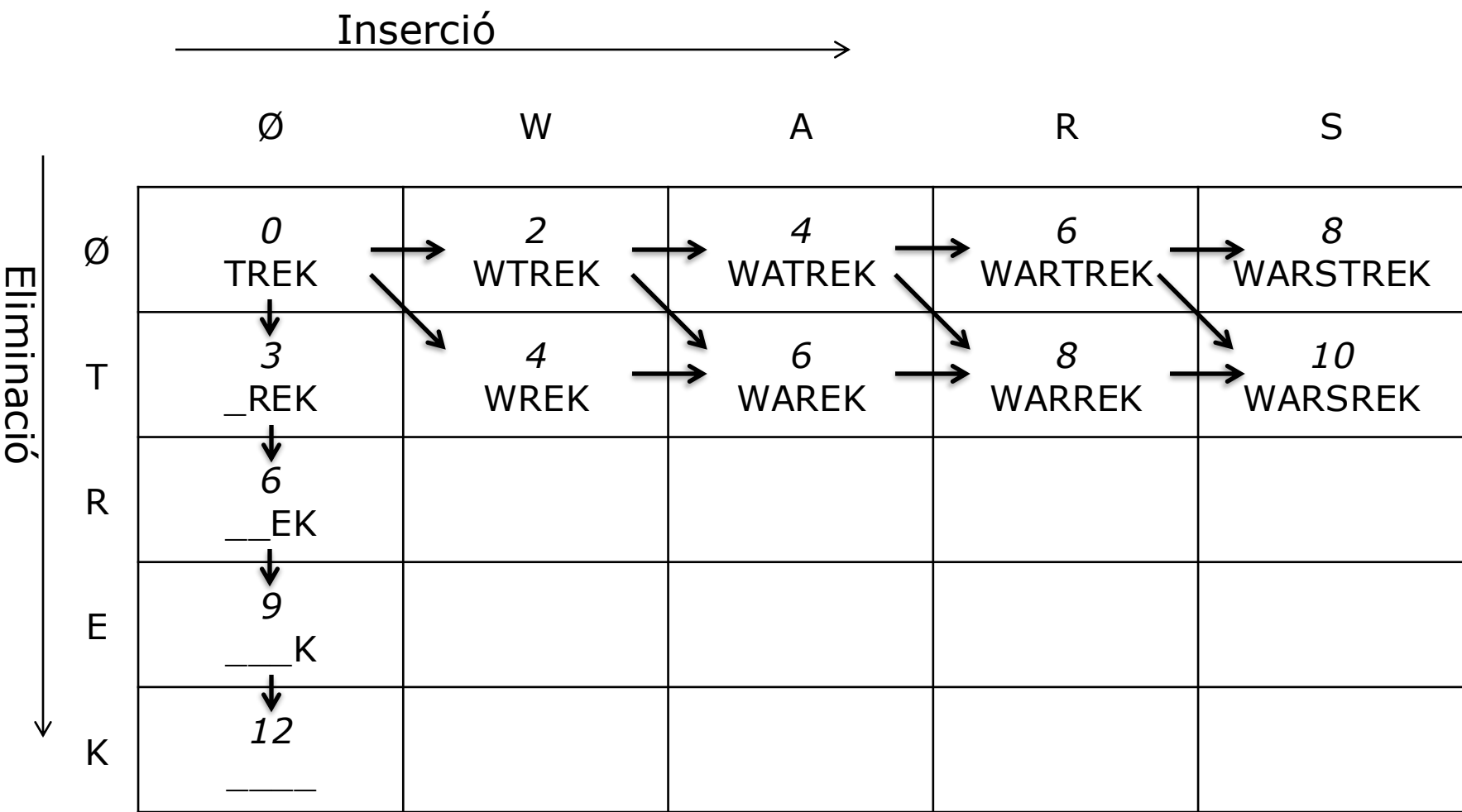
Reconeixement estructural

Pas 3: Calcular els costos de la columna 1



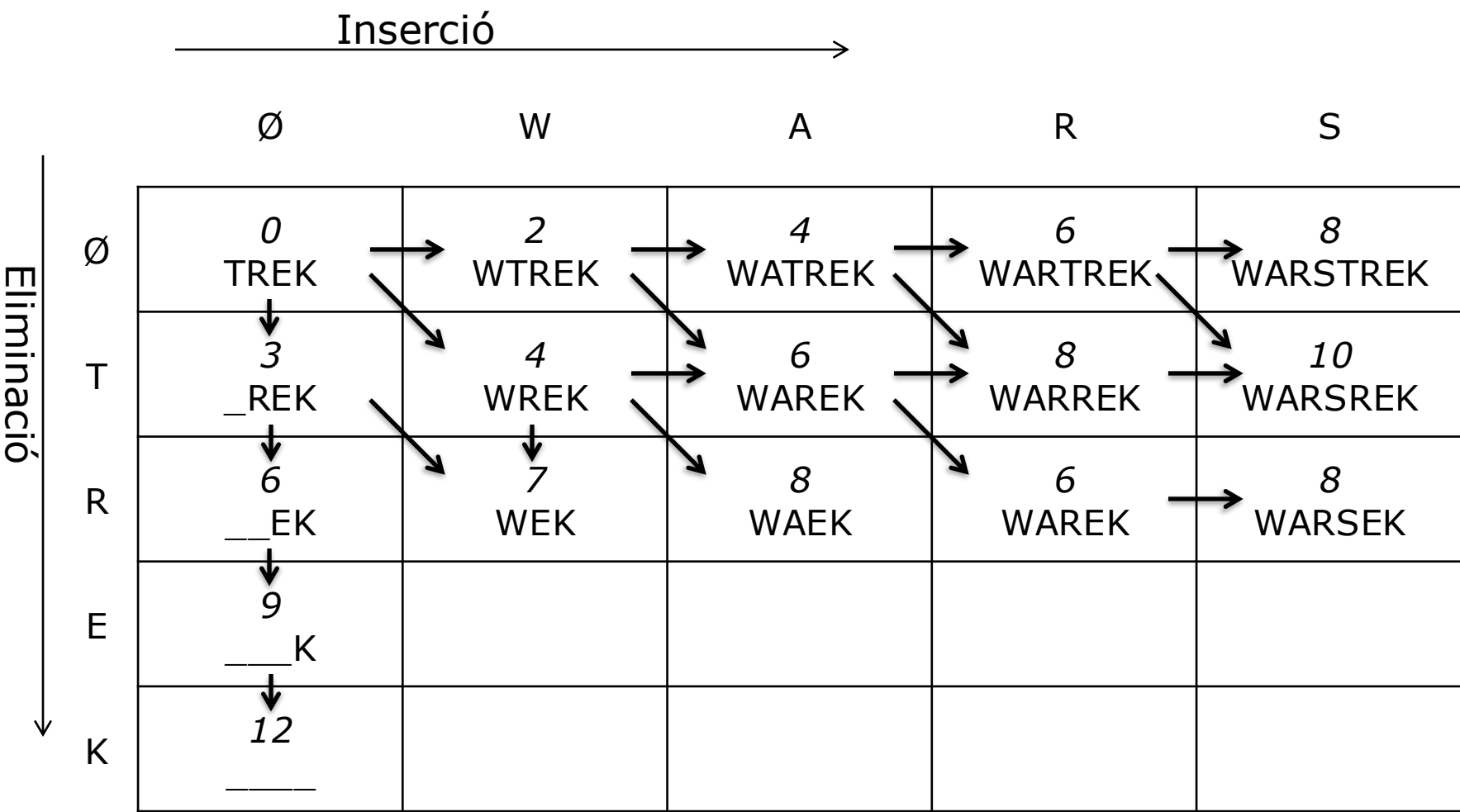
Reconeixement estructural

Pas 4: Calcular la resta de costos d'esquerra a dreta i de dalt abaix aplicant l'operació de mínim.



Reconeixement estructural

Pas 4: Calcular la resta de costos d'esquerra a dreta i de dalt abaix aplicant l'operació de mínim.



Reconeixement estructural

Pas 4: Calcular la resta de costos d'esquerra a dreta i de dalt abaix aplicant l'operació de mínim.

Inserció →

	∅	W	A	R	S
∅	0 TREK	2 WTREK	4 WATREK	6 WARTREK	8 WARSTREK
T	3 _REK	4 WREK	6 WAREK	8 WARREK	10 WARSREK
R	6 __EK	7 WEK	8 WAEK	6 WAREK	8 WARSEK
E	9 ___K	10 WK	11 WAK	9 WARK	10 WARSK
K	12 ____				

Eliminació ↓

Reconeixement estructural

Pas 4: Calcular la resta de costos d'esquerra a dreta i de dalt abaix aplicant l'operació de mínim.

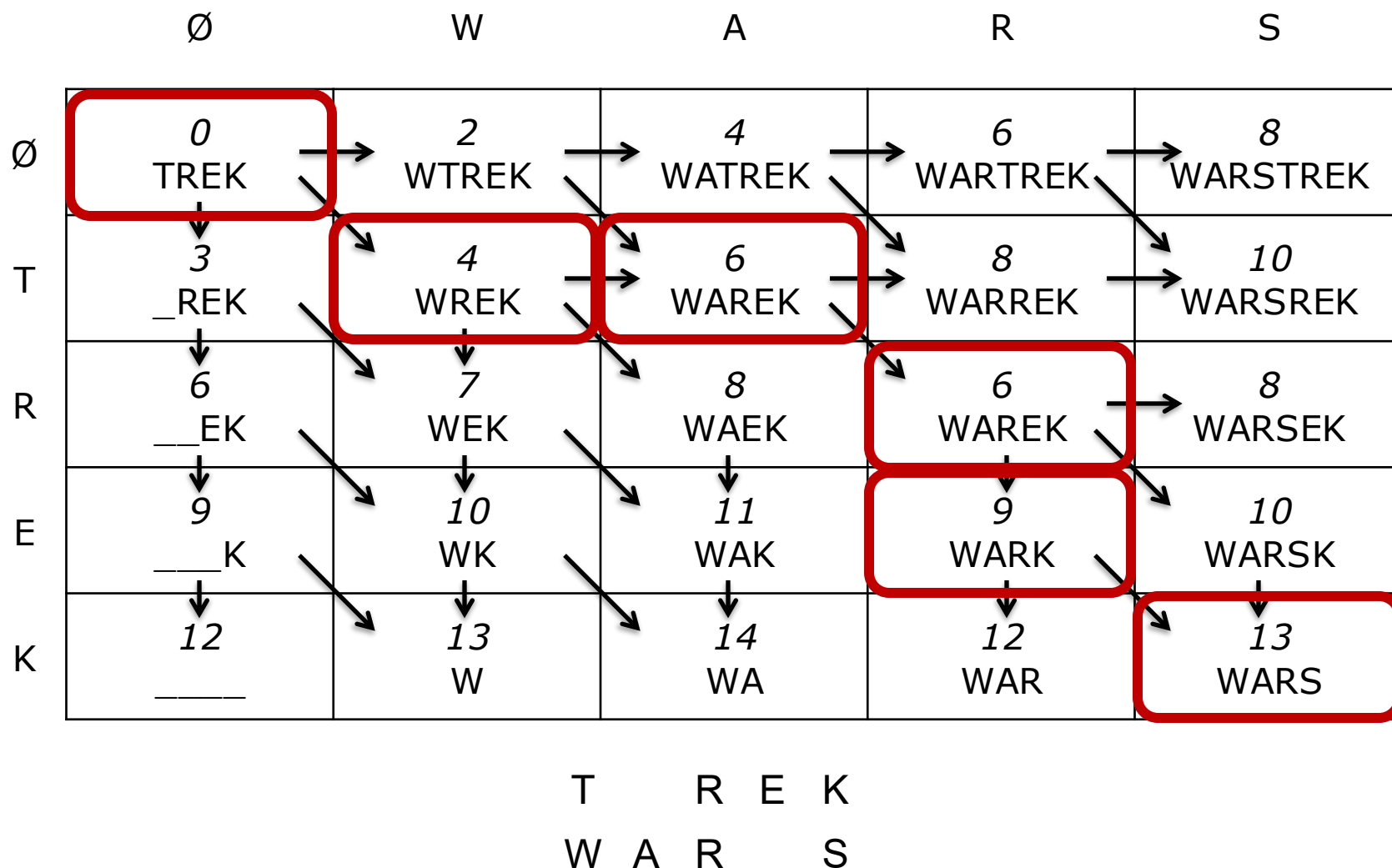
Inserció →

	∅	W	A	R	S
∅	0 TREK	2 WTREK	4 WATREK	6 WARTREK	8 WARSTREK
T	3 _REK	4 WREK	6 WAREK	8 WARREK	10 WARSREK
R	6 __EK	7 WEK	8 WAEK	6 WAREK	8 WARSEK
E	9 ___K	10 WK	11 WAK	9 WARK	10 WARSK
K	12 ____	13 W	14 WA	12 WAR	13 WARS

↓ Eliminació

Reconeixement estructural

Pas 5: Extreure la seqüència d'operacions d'edició que ens porten a la distància d'edició mínima



Reconeixement estructural

Pas 5: Extreure la seqüència d'operacions d'edició que ens porten a la distància d'edició mínima

	∅	W	A	R	S
∅	0 TREK	2 WTREK	4 WATREK	6 WARTREK	8 WARSTREK
T	3 _REK	4 WREK	6 WAREK	8 WARREK	10 WARSREK
R	6 __EK	7 WEK	8 WAEK	6 WAREK	8 WARSEK
E	9 ___K	10 WK	11 WAK	9 WARK	10 WARSK
K	12 ____	13 W	14 WA	12 WAR	13 WARS

T R E K
W A R S

Reconeixement estructural

Pas 5: Extreure la seqüència d'operacions d'edició que ens porten a la distància d'edició mínima

	∅	W	A	R	S
∅	0 TREK	2 WTREK	4 WATREK	6 WARTREK	8 WARSTREK
T	3 _REK	4 WREK	6 WAREK	8 WARREK	10 WARSREK
R	6 __EK	7 WEK	8 WAEK	6 WAREK	8 WARSEK
E	9 ___K	10 WK	11 WAK	9 WARK	10 WARSK
K	12 ____	13 W	14 WA	12 WAR	13 WARS

T R E K
W A R S

Reconeixement estructural

Problema 4: Cerca de perfils d'usuaris en xarxes socials. Suposem que els usuaris d'una xarxa social estan classificats en funció de tres categories segons si els agrada la música, l'esport o la tecnologia. I per a cada usuari coneixem tots els usuaris al que segueix.

Com podem solucionar el problema de cercar certs perfils d'usuaris dins de la xarxa social?

Exemple: "Cercar els usuaris que els agrada la música i són seguidors de com a mínim dos usuaris que els agrada l'esport"

Contesteu a les següents preguntes sobre la resolució d'aquest problema com un problema de reconeixement de patrons estructural?

1. Què són els nodes?, què són els arcs?
2. Quins grafs hi ha?
3. Quina estratègia/algorismes faries servir per fer una cerca d'aquest estil?
4. Es pot fer servir l'algorisme AC amb alguna propietat?

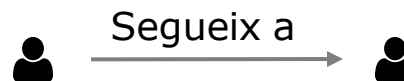
Reconeixement estructural

1. Què són els nodes?, què són els arcs?

El nodes són els usuaris, que n'hi ha de 3 tipus segons el que els agrada:

-  : Usuari que li agrada la música
-  : Usuari que li agrada l'esport
-  : Usuari que li agrada la tecnologia

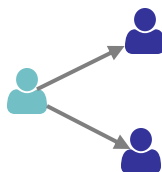
Els arcs indiquen que un usuari en segueix un altre:



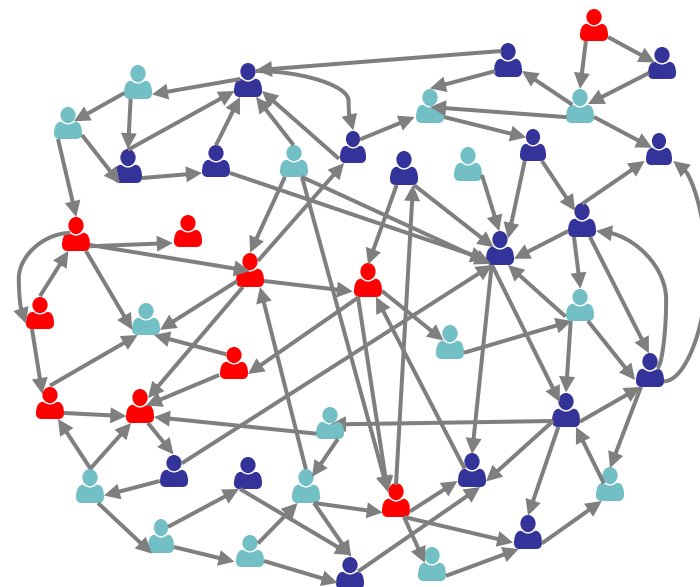
2. Quins grafs hi ha?

GRAF 1: graf de la cerca que volem fer

Exemple: "Cercar els usuaris que els agrada la música i són seguidors de com a mínim dos usuaris que els agrada l'esport"



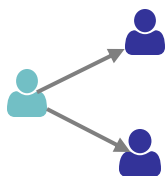
GRAF 2: la xarxa social



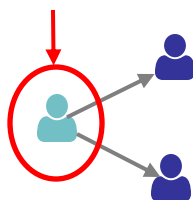
Reconeixement estructural

3. Quina estratègia/algorismes faries servir per fer la cerca?

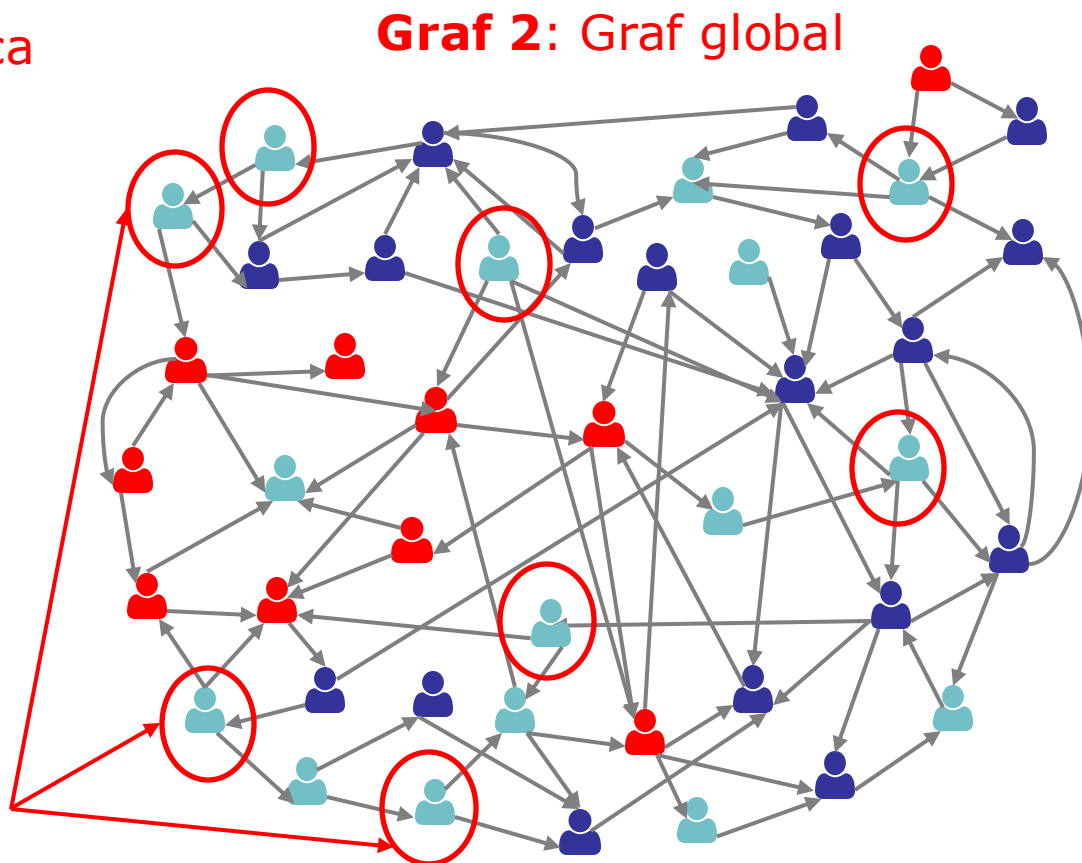
Graf 1: Graf que es cerca



Pas 1. Identificar el node principal del Graf 1



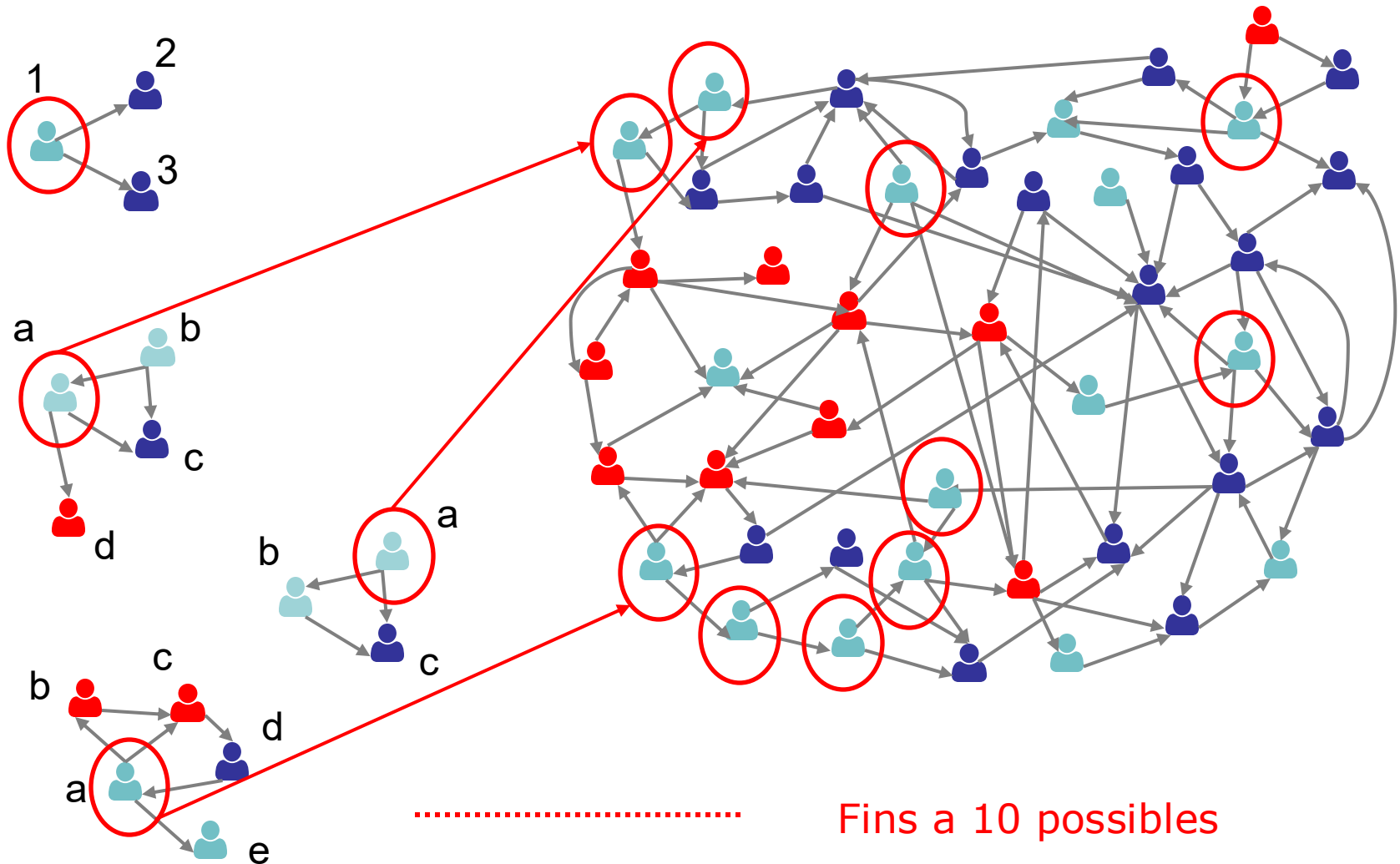
Pas 2. Identificar nodes semblants al Graf 2.



Pas 3. Per a cada un fer un graph-matching des del node principal cap a la resta

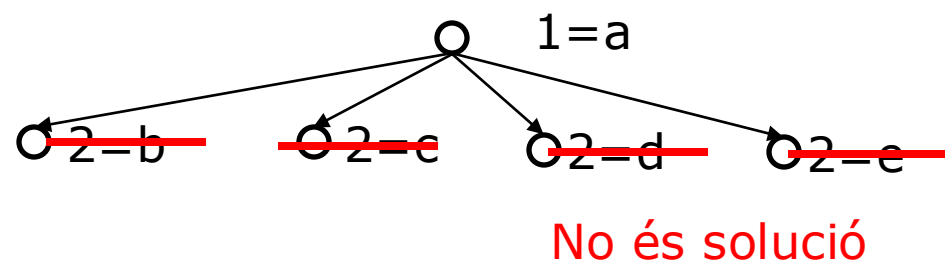
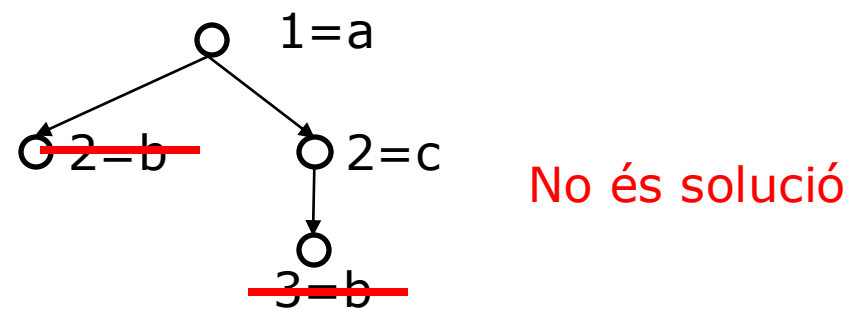
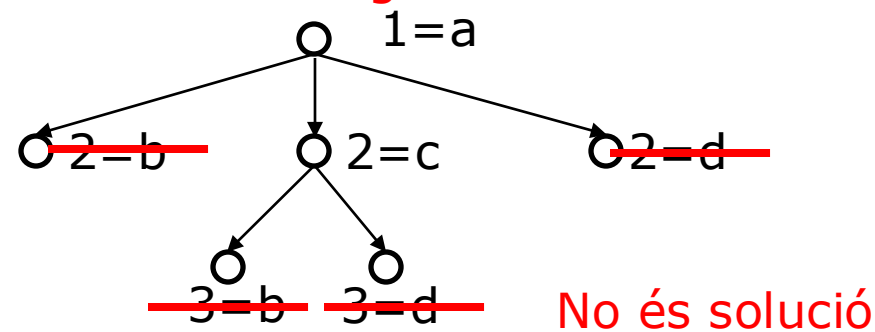
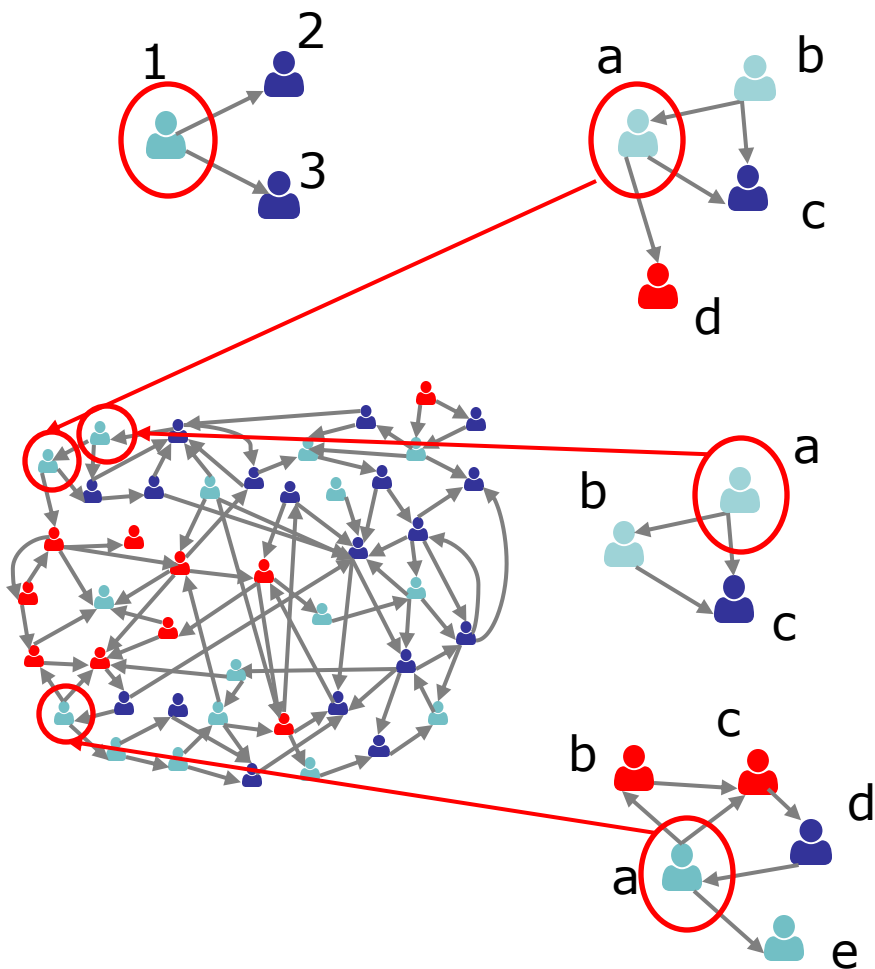
Reconeixement estructural

Pas 3. Per a cada un fer un correspondència des del node principal cap a la resta. Cal considerar tipus de node i direcció de l'enllaç.



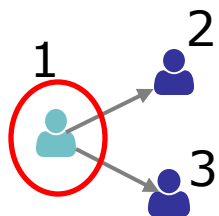
Reconeixement estructural

Pas 3. Per fer la correspondència el millor és una estratègia de backtracking que va comprovant les restriccions del graf inicial

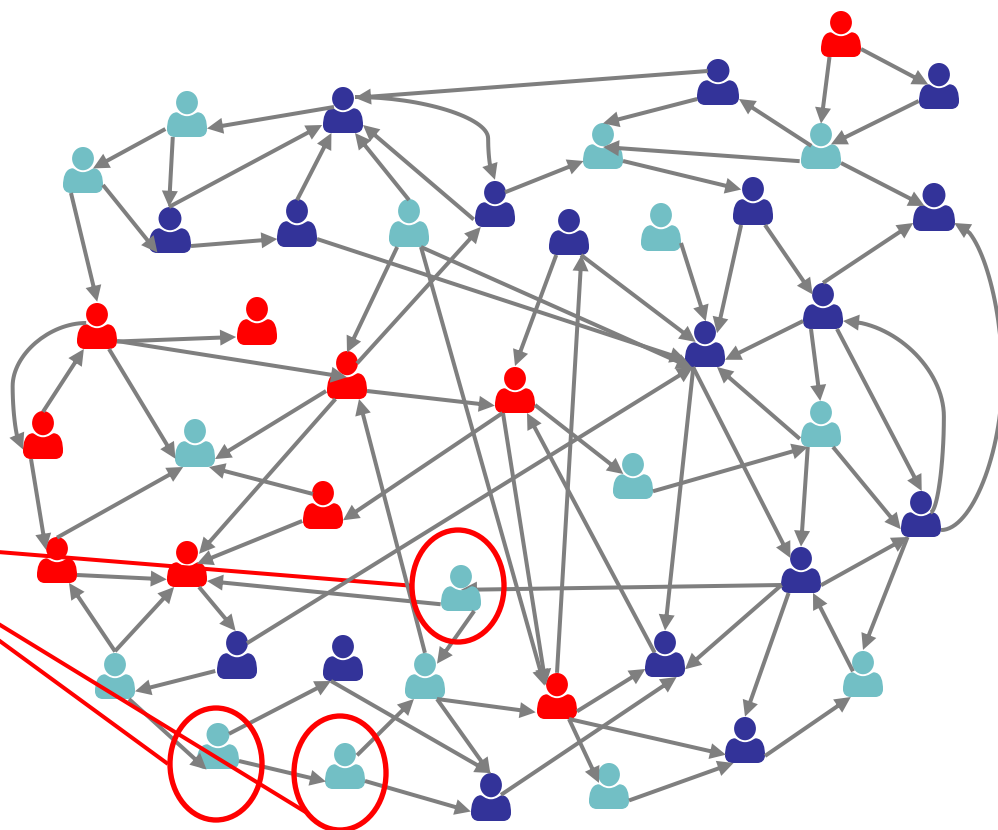


Reconeixement estructural

Pas 3. Per a cada un fer un graph-matching des del node principal cap a la resta. Cal considerar tipus de node i direcció de l'enllaç.

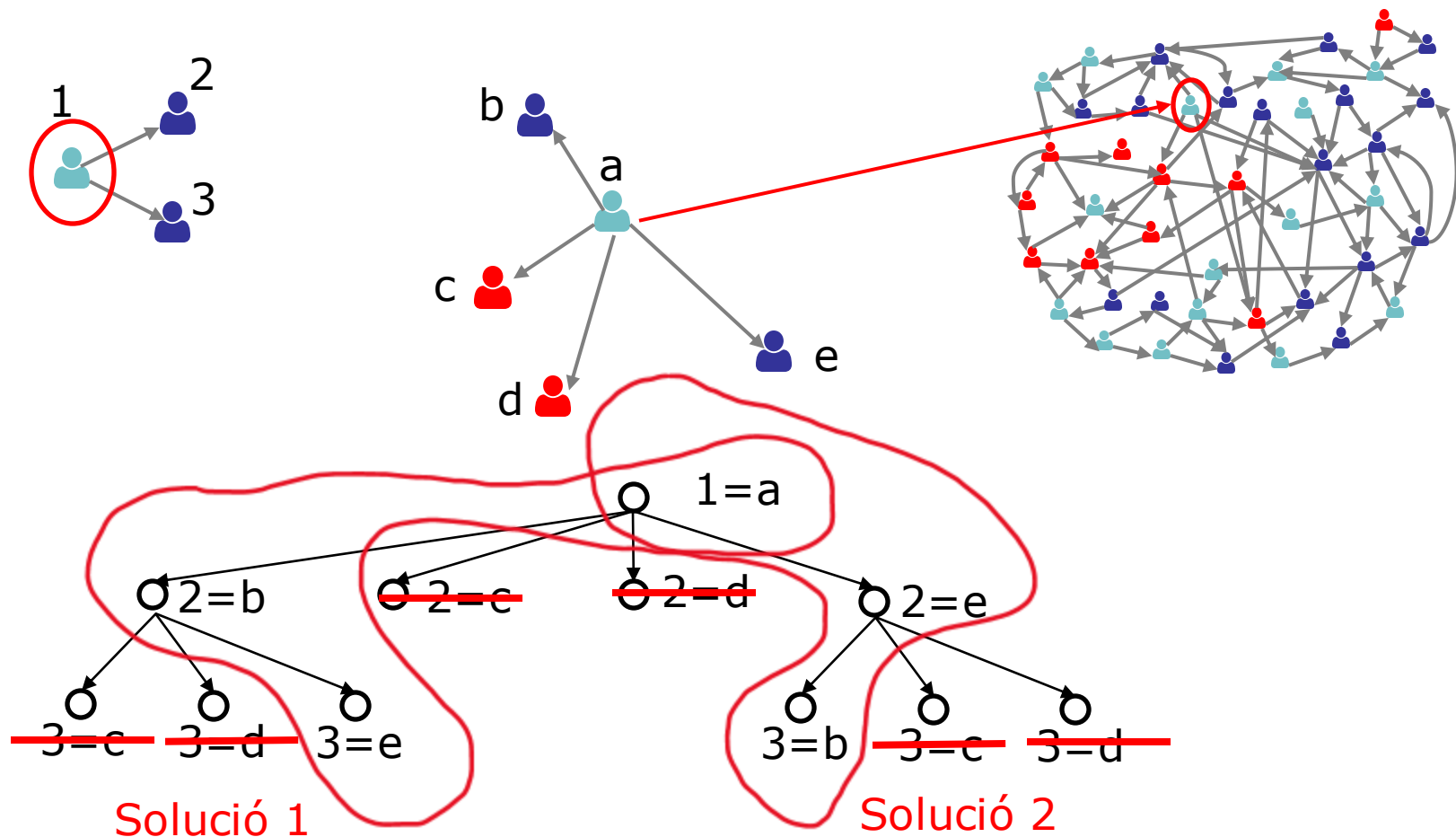


Tampoc són solució



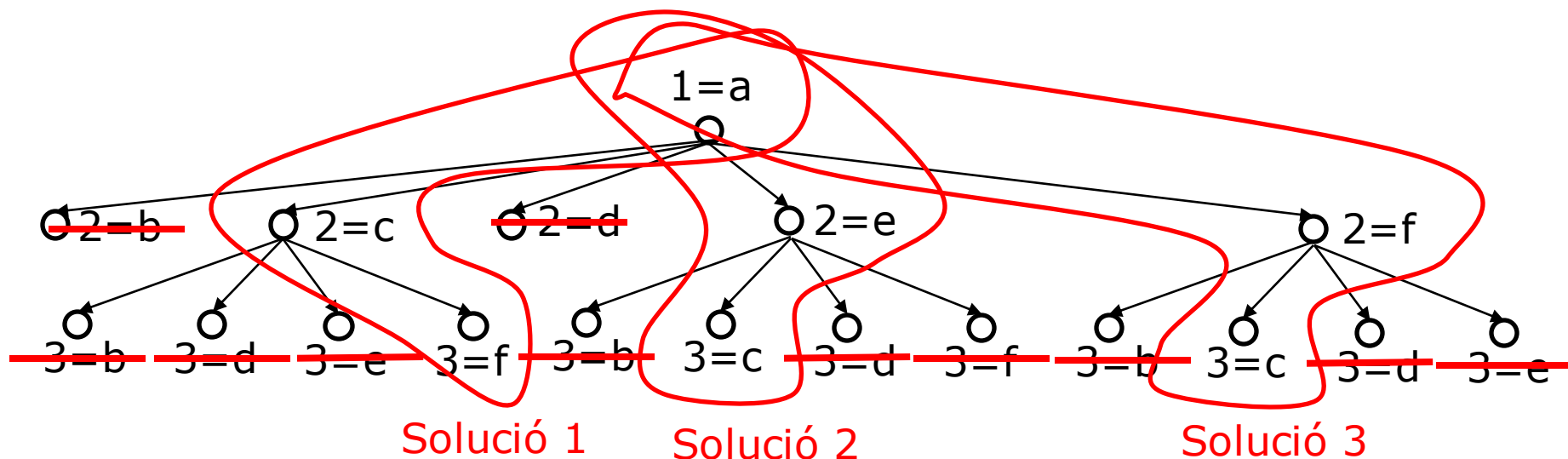
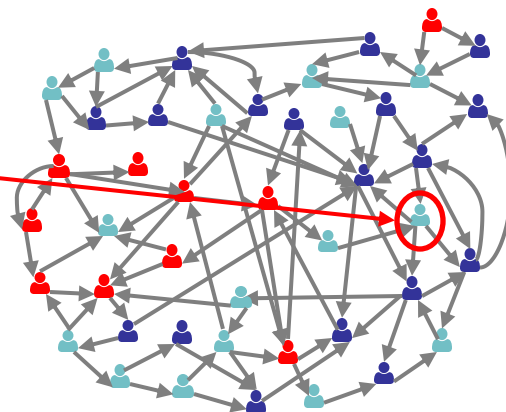
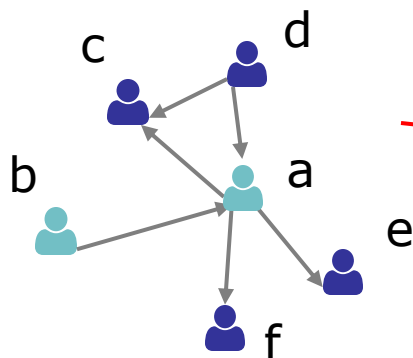
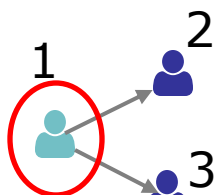
Reconeixement estructural

Pas 3. Per a cada un fer un graph-matching des del node principal cap a la resta. Cal considerar tipus de node i direcció de l'enllaç.



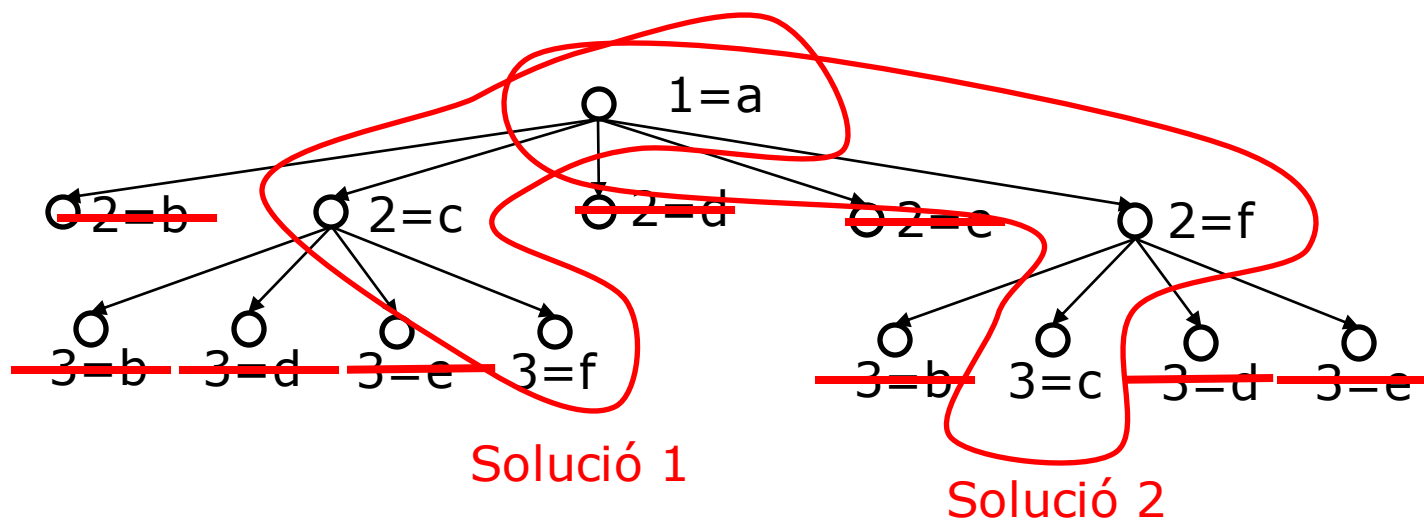
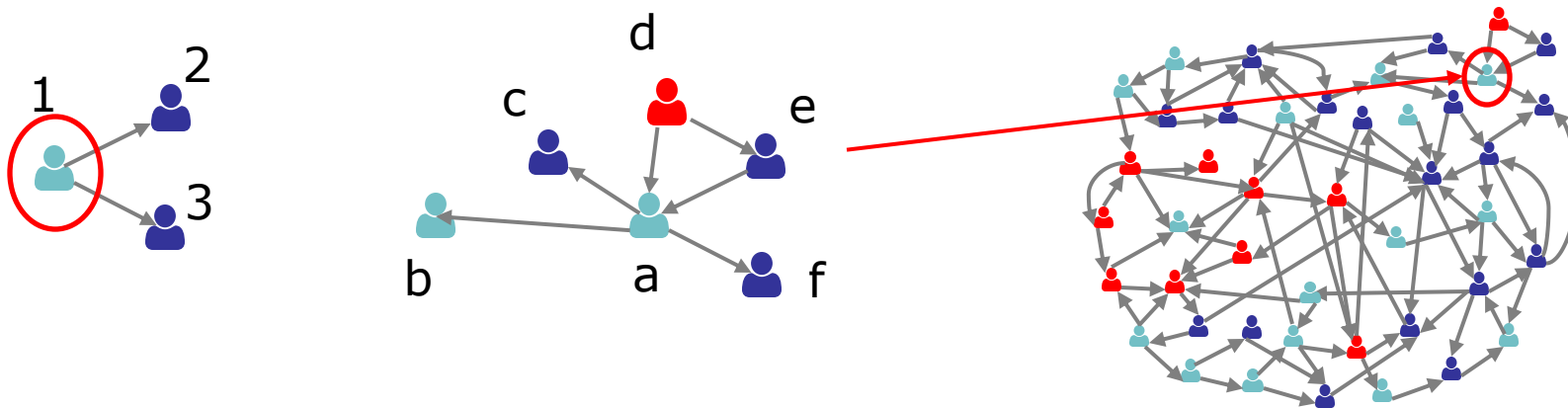
Reconeixement estructural

Pas 3. Per a cada un fer un graph-matching des del node principal cap a la resta. Cal considerar tipus de node i direcció de l'enllaç.



Reconeixement estructural

Pas 3. Per a cada un fer un graph-matching des del node principal cap a la resta. Cal considerar tipus de node i direcció de l'enllaç.



Reconeixement estructural

4. Es pot fer servir l'algorisme AC amb alguna propietat?

Sí, amb el tipus de node dels que estan connectats al node principal

Exemple:

